Dresden University of Technology

Computational Logic ∴ Group

Sarah Gaggl

Faculty of Computer Science, Institute of Artificial Intelligence, Computational Logic Group

Uninformed Search vs. Informed Search

Lecture 2 // Advanced Problem Solving and Search, WS 2026/27

Agenda

1. Introduction
2. Uninformed Search versus Informed Search (Best First Search, A* Search, Dynamic Programming)
3. Local Search, Stochastic Hill Climbing, Simulated Annealing
4. Tabu Search
5. Evolutionary Algorithms/ Genetic Algorithms
6. Answer-set Programming (ASP)
7. Constraint Satisfaction (CSP)
8. Structural Decomposition Techniques (Tree/Hypertree Decompositions)

Traditional Methods

- There are many classic algorithms to search spaces for an optimal solution.
- Broadly, they fall into two disjoint classes:
  - Algorithms that only evaluate complete solutions (exhaustive search, local search, . . .).
  - Algorithms that require the evaluation of partially constructed or approximate solutions.
- Algorithms that treat complete solutions can be stopped any time, and give at least one potential answer.
- If you interrupt an algorithm that works on partial solutions, the results might be useless.

Complete Solutions

- All decision variables are specified.
- For example, binary strings of length *n* constitute complete solutions for any *n*-variable SAT.
- Permutations of *n* cities constitute complete solutions for a TSP.
- We can compare two complete solutions using an evaluation function.
- Many algorithms rely on such comparisons, manipulating one single complete solution at a time.
- When a new solution has a better evaluation than the previous best solution, it replaces that prior solution.
- Exhaustive search, local search, hill climbing as well as modern heuristic methods such as simulated annealing, tabu search and evolutionary algorithms fall into this category.

Partial Solutions

There are two forms:

1. incomplete solution to the problem originally posed, and
2. complete solution to a reduced (i.e. simpler) problem.

- Incomplete solutions reside in a subset of the original problem's search space.
  - In an SAT, consider all of the binary strings where the first two variables were assigned the value 1 (i.e. TRUE).
  - In a TSP, consider every permutation of cities that contains the sequence $7 - 11 - 2 - 16$.
  - We fix the attention on a subset of the search space that has a partial property.
  - Hopefully, that property is also shared by the real solution!

Partial Solutions ctd.

- Decompose original problem into a set of smaller and simpler problems.
  - Hope: solving each of the easier problems and combine the partial solutions, results in an answer for the original problem.
  - In a TSP, consider only k out of n cities and try to establish the shortest path from city i to j that passes through all k of these cities.
  - Reduce the size of the search space significantly and search for a complete solution within the restricted domain.
  - Such partial solutions can serve as building blocks for the solution to the original problem.

Partial Solutions ctd.

- Decompose original problem into a set of smaller and simpler problems.
  - Hope: solving each of the easier problems and combine the partial solutions, results in an answer for the original problem.
  - In a TSP, consider only k out of n cities and try to establish the shortest path from city i to j that passes through all k of these cities.
  - Reduce the size of the search space significantly and search for a complete solution within the restricted domain.
  - Such partial solutions can serve as building blocks for the solution to the original problem.
- But, algorithms that work on partial solutions pose additional difficulties. One needs to
  - devise a way to organize the sub-spaces so that they can be searched efficiently, and
  - create a new evaluation function that can assess the quality of partial solutions.

Exhaustive Search

- Checks every solution in the search space until the best global solution has been found.
- Can be used only for small instances of problems.
- Exhaustive (enumerative) algorithms are simple.
- Search space can be reduced by backtracking.
- Some optimization methods, e.g., branch and bound and A* are based on an exhaustive search.

Exhaustive Search

- Checks every solution in the search space until the best global solution has been found.
- Can be used only for small instances of problems.
- Exhaustive (enumerative) algorithms are simple.
- Search space can be reduced by backtracking.
- Some optimization methods, e.g., branch and bound and A* are based on an exhaustive search.
- How can we generate a sequence of every possible solution to the problem?
  - The order in which the solutions are generated and evaluated is irrelevant (because we evaluate all of them).
  - The answer for the question depends on the selected representation.

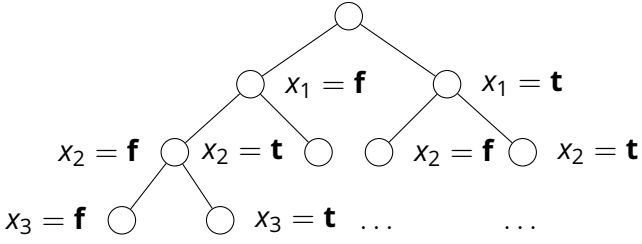
Enumerating the SAT

- We have to generate every possible binary string of length n.
- All solutions correspond to whole numbers in a one-to-one mapping.
- Generate all non-negative integers from 0 to $2^n - 1$ and convert each of these integers into the matching binary string of length n.

0000	0	0100	4	1000	8	1100	12
0001	1	0101	5	1001	9	1101	13
0010	2	0110	6	1010	10	1110	14
0011	3	0111	7	1011	11	1111	15

- Bits of the string are the truth assignments of the decision variables.
- Organize the search space, for example partition into two disjoint sub-spaces. First contains all the vectors where $x_1 = \mathbf{f}$ (FALSE), and the second contains all vectors where $x_1 = \mathbf{t}$ (TRUE).

Enumerating the SAT ctd.



Binary search tree for SAT

Search Strategies

A strategy is defined by picking the order of node expansion.
Strategies are evaluated along the following dimensions:

- Completeness - does it always find a solution if one exists?
- Time complexity - number of nodes generated/expanded.
- Space complexity - maximum number of nodes in memory.
- Optimality - does it always find a least-cost solution?

Time and space complexity are measured in terms of

- b - maximum branching factor of the search tree;
- d - depth of the least-cost solution;
- m - maximum depth of the state space (may be ∞).

Uninformed Search

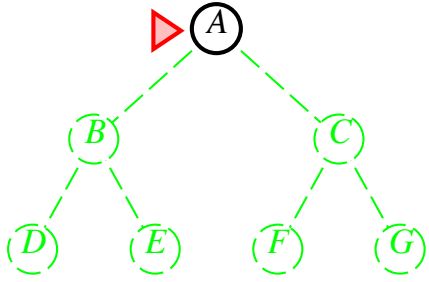
Uninformed Search Strategies

Uninformed strategies use only the information available in the problem definition.

- Breadth-first search
- Depth-first search
- Depth-limited search
- Iterative deepening search

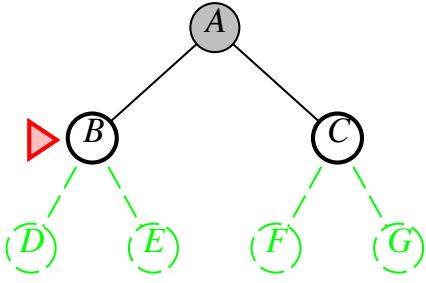
Breadth-First Search

- Expand shallowest unexpanded node.
- FIFO queue, i.e. new nodes go to the back of the queue, and old nodes get expanded first.



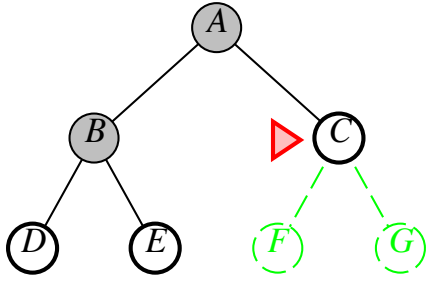
Breadth-First Search

- Expand shallowest unexpanded node.
- FIFO queue, i.e. new nodes go to the back of the queue, and old nodes get expanded first.



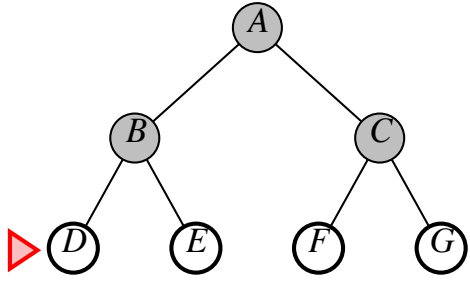
Breadth-First Search

- Expand shallowest unexpanded node.
- FIFO queue, i.e. new nodes go to the back of the queue, and old nodes get expanded first.



Breadth-First Search

- Expand shallowest unexpanded node.
- FIFO queue, i.e. new nodes go to the back of the queue, and old nodes get expanded first.



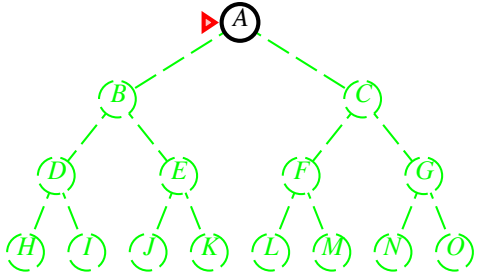
Properties of breadth-first search

- Complete: Yes (if b is finite)
- Time: $1 + b + b^2 + b^3 + \ldots + b^d + b(b^d - 1) = O(b^{d+1})$, i.e., exp. in d
- Space: $O(b^{d+1})$ (keeps every node in memory)
- Optimal: Yes (if cost = 1 per step); not optimal in general

Space is the big problem; can easily generate nodes at 100MB/sec so 24hrs = 8640GB.

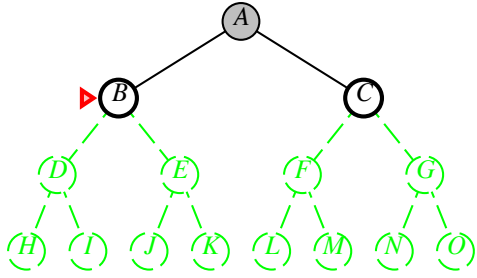
Depth-First Search

- Expand deepest unexpanded node.
- LIFO queue, i.e. most recently generated node is chosen for expansion.



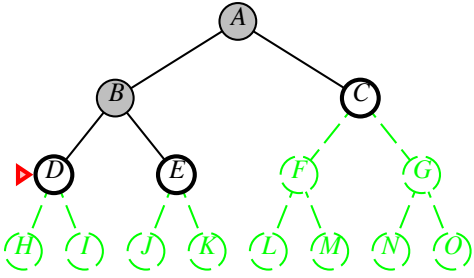
Depth-First Search

- Expand deepest unexpanded node.
- LIFO queue, i.e. most recently generated node is chosen for expansion.



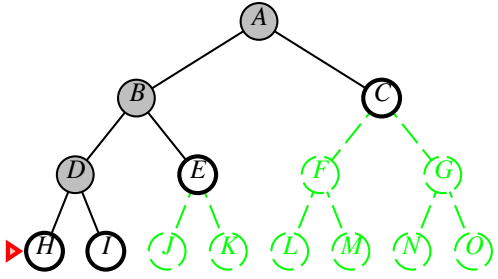
Depth-First Search

- Expand deepest unexpanded node.
- LIFO queue, i.e. most recently generated node is chosen for expansion.



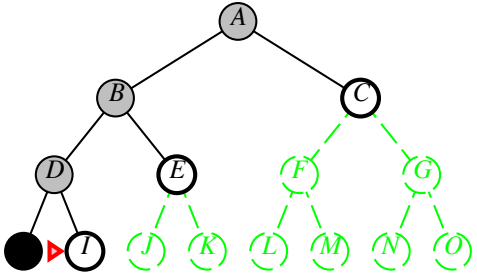
Depth-First Search

- Expand deepest unexpanded node.
- LIFO queue, i.e. most recently generated node is chosen for expansion.



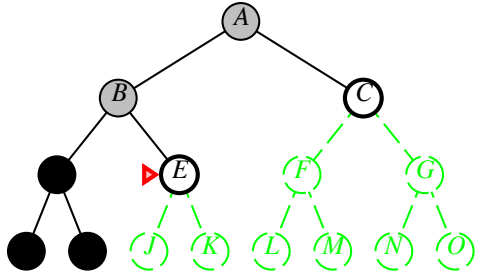
Depth-First Search

- Expand deepest unexpanded node.
- LIFO queue, i.e. most recently generated node is chosen for expansion.



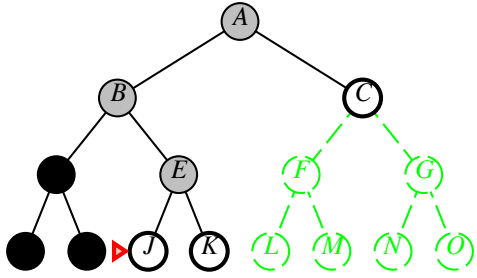
Depth-First Search

- Expand deepest unexpanded node.
- LIFO queue, i.e. most recently generated node is chosen for expansion.



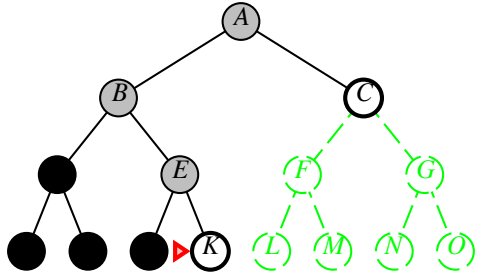
Depth-First Search

- Expand deepest unexpanded node.
- LIFO queue, i.e. most recently generated node is chosen for expansion.



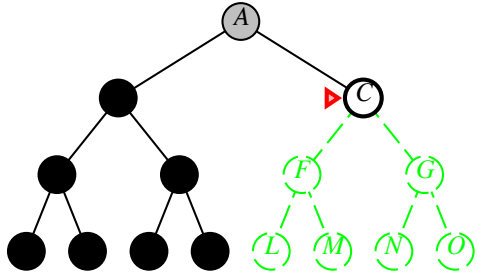
Depth-First Search

- Expand deepest unexpanded node.
- LIFO queue, i.e. most recently generated node is chosen for expansion.



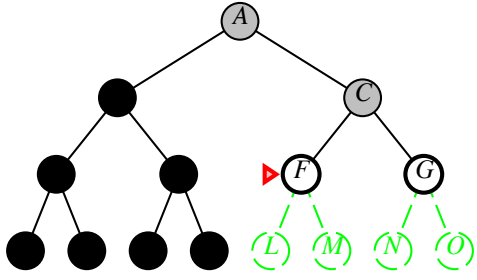
Depth-First Search

- Expand deepest unexpanded node.
- LIFO queue, i.e. most recently generated node is chosen for expansion.



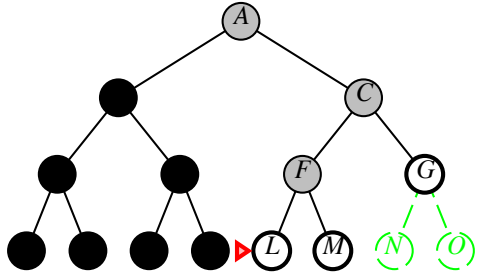
Depth-First Search

- Expand deepest unexpanded node.
- LIFO queue, i.e. most recently generated node is chosen for expansion.



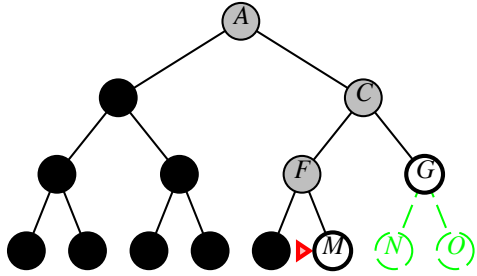
Depth-First Search

- Expand deepest unexpanded node.
- LIFO queue, i.e. most recently generated node is chosen for expansion.



Depth-First Search

- Expand deepest unexpanded node.
- LIFO queue, i.e. most recently generated node is chosen for expansion.

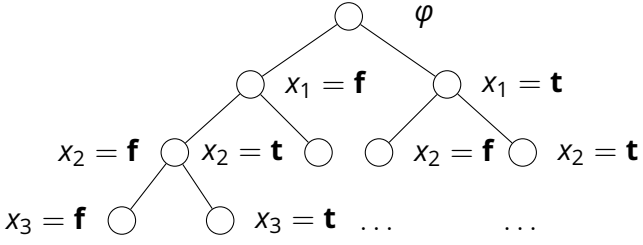


Properties of Depth-First Search

- Complete: No: fails in infinite-depth spaces, spaces with loops
- Time: $O(b^m)$: terrible if m is much larger than d;
  but if solutions are dense, may be much faster than breadth-first
- Space: $O(bm)$, i.e., linear space!
- Optimal: No

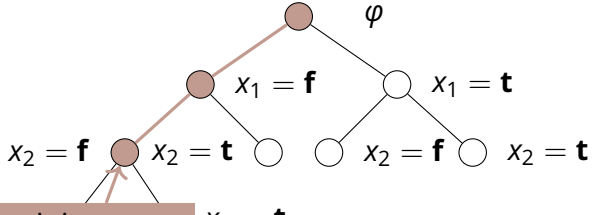
Backtracking

Suppose the SAT formula φ contains a clause $(x_1 \vee x_2)$.



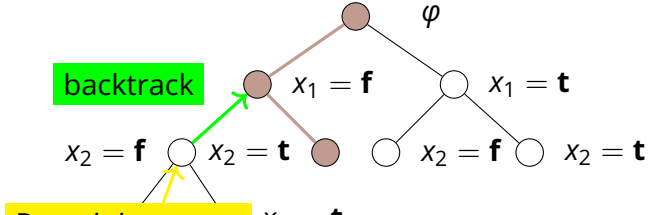
Backtracking

Suppose the SAT formula φ contains a clause $(x_1 \vee x_2)$.



Backtracking

Suppose the SAT formula φ contains a clause $(x_1 \vee x_2)$.



Depth-Limited Search

- Depth first search with depth limit L
  - Nodes at depth L are not expanded.
- Eliminates problem with infinite path.
- How to select L?
- Possible failures:
  - No solution;
  - Cutoff - no solution within the depth limit.

Iterative Deepening Search

Repeat Depth-limited search with L=1,2,3,...



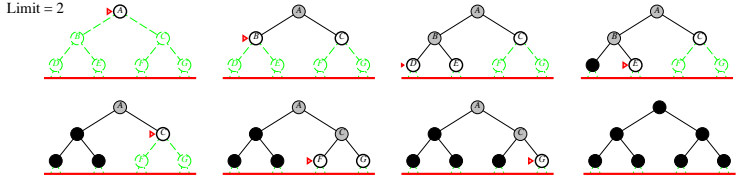
Iterative Deepening Search

Repeat Depth-limited search with L=1,2,3,...



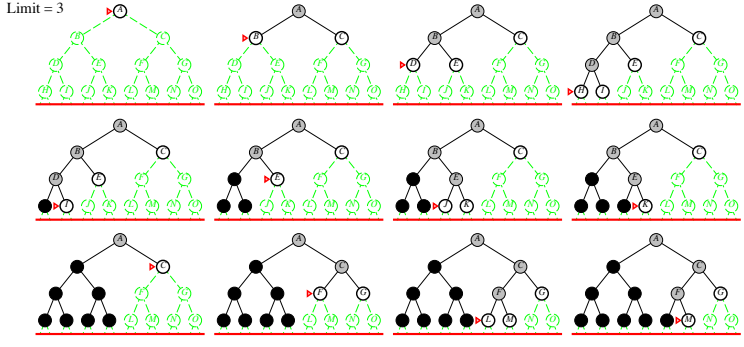
Iterative Deepening Search

Repeat Depth-limited search with L=1,2,3,...



Iterative Deepening Search

Repeat Depth-limited search with L=1,2,3,...



Properties of Iterative Deepening Search

- Complete: Yes
- Time: $(d+1)b^0 + db^1 + (d-1)b^2 + \ldots + b^d = O(b^d)$
- Space: $O(bd)$
- Optimal: Yes, if step cost = 1

Properties of Iterative Deepening Search

- Complete: Yes
- Time: $(d+1)b^0 + db^1 + (d-1)b^2 + \ldots + b^d = O(b^d)$
- Space: $O(bd)$
- Optimal: Yes, if step cost = 1

Number of nodes generated in worst case for $b = 10$ and $d = 5$ (solution at far right leaf):

$$N(\text{IDS}) = 50 + 400 + 3,000 + 20,000 + 100,000 = 123,450$$
$$N(\text{BFS}) = 10 + 100 + 1,000 + 10,000 + 100,000 = 111,100$$

Hybrid approach that runs BFS until almost all memory is consumed, and then runs IDS from all the nodes in the frontier.
In general, IDS is the preferred uninformed search method when the search space is large and the depth of the solution is not known.

Informed Search

Best-First Search

Idea: use an evaluation function for each node

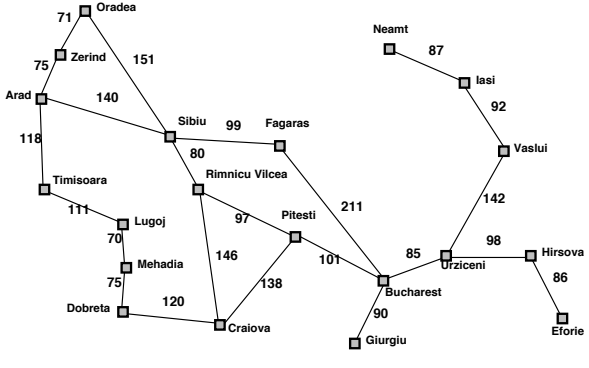
- estimate of “desirability”

$\Rightarrow$ Expand most desirable unexpanded node

Special cases:

- Greedy search
- A* search
- Dynamic programming

Example: Romania with step costs in km



Straight–line distance to Bucharest

Arad	366
Bucharest	0
Craiova	160
Dobreta	242
Eforie	161
Fagaras	178
Giurgiu	77
Hirsova	151
Iasi	226
Lugoj	244
Mehadia	241
Neamt	234
Oradea	380
Pitesti	98
Rimnicu Vilcea	193
Sibiu	253
Timisoara	329
Urziceni	80
Vaslui	199
Zerind	374

Greedy Search

- Evaluation function $h(n)$ (**h**euristic)
  $h(n)$ = estimate of cost from n to the closest goal
- E.g., $h_{SLD}(n)$ = straight-line distance from n to Bucharest
- Greedy search expands the node that **appears** to be closest to goal

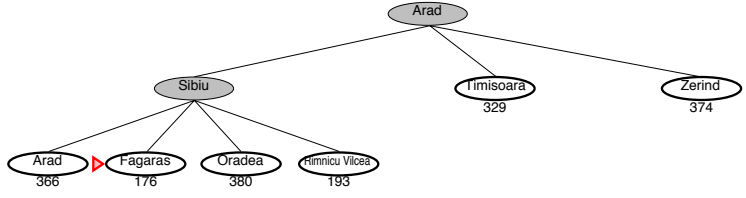
Greedy Search Example



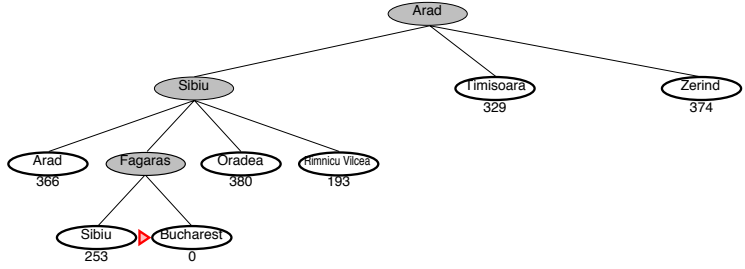
Greedy Search Example



Greedy Search Example



Greedy Search Example



Properties of Greedy Search

- Complete: No–can get stuck in loops, e.g.,
  - Iasi → Neamt → Iasi → Neamt →
  - Complete in finite space with repeated-state checking
- Time: $O(b^m)$, but a good heuristic can give dramatic improvement
- Space: $O(b^m)$—keeps all nodes in memory
- Optimal: No

A* Search

Idea: avoid expanding paths that are already expensive

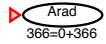
- Evaluation function $f(n) = g(n) + h(n)$

  $g(n)$ = cost so far to reach n
  $h(n)$ = estimated cost to goal from n
  $f(n)$ = estimated total cost of path through n to goal

- A* search uses an admissible heuristic
  - i.e., $h(n) \leq h^*(n)$ where $h^*(n)$ is the **true** cost from n.
  - Also require $h(n) \geq 0$, so $h(G) = 0$ for any goal G.
- E.g., $h_{SLD}(n)$ never overestimates the actual road distance

Theorem: A* search is optimal

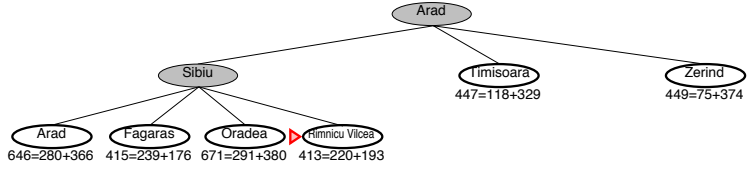
A* Search Example



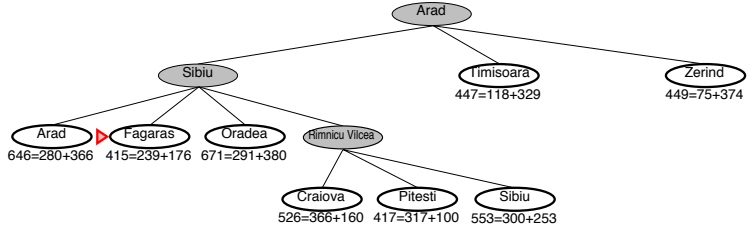
A* Search Example



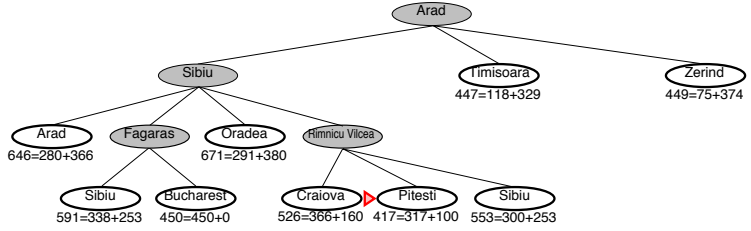
A* Search Example



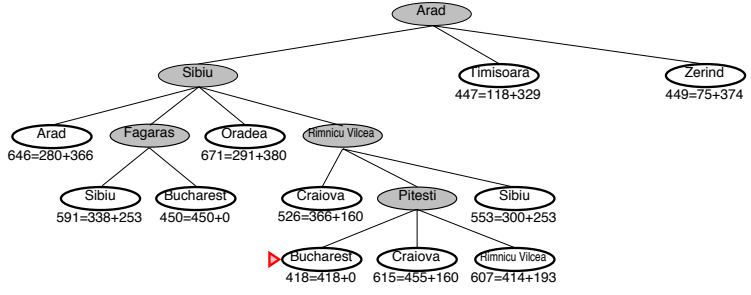
A* Search Example



A* Search Example

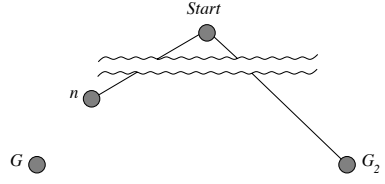


A* Search Example



Optimality of A* (standard proof)

Suppose some suboptimal goal G_2 has been generated and is in the queue. Let n be an unexpanded node on a shortest path to an optimal goal G.



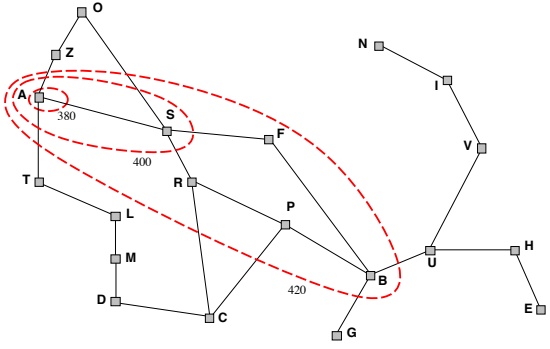
$$
\begin{aligned}
f(G_2) &= g(G_2) && \text{since } h(G_2) = 0 \\
&> g(G) && \text{since } G_2 \text{ is suboptimal} \\
&\geq f(n) && \text{since } h \text{ is admissible}
\end{aligned}
$$

Since $f(G_2) > f(n)$, A* will never select G_2 for expansion

Optimality of A* (more useful)

Lemma: A* expands nodes in order of increasing f value*

- Gradually adds "f-contours" of nodes (cf. breadth-first adds layers)
- Contour i has all nodes with $f = f_i$, where $f_i < f_{i+1}$



Properties of A*

- Complete: Yes, unless there are infinitely many nodes with $f \leq f(G)$
- Time: Exponential in [relative error in $h \times$ length of soln.]
- Space: Keeps all nodes in memory
- Optimal: Yes—cannot expand f_{i+1} until f_i is finished
- A* expands all nodes with $f(n) < C^*$
- A* expands some nodes with $f(n) = C^*$
- A* expands no nodes with $f(n) > C^*$

where C^* is cost for optimal solution

Dynamic Programming

The Problem with Naive Recursion

- Many recursive definitions are straightforward (e.g. Fibonacci, subset sum)
- But naive recursion often recomputes the same sub-problems over and over
- This leads to exponential time in many cases

The Problem with Naive Recursion

- Many recursive definitions are straightforward (e.g. Fibonacci, subset sum)
- But naive recursion often recomputes the same sub-problems over and over
- This leads to exponential time in many cases

Fibonacci numbers

The Problem with Naive Recursion

- Many recursive definitions are straightforward (e.g. Fibonacci, subset sum)
- But naive recursion often recomputes the same sub-problems over and over
- This leads to exponential time in many cases

Fibonacci numbers

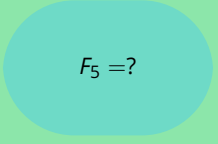
$F_n := F_{n-1} + F_{n-2}$ for $n > 2$
Base case: $F_1 := 1$ and $F_2 := 1$

The Problem with Naive Recursion

- Many recursive definitions are straightforward (e.g. Fibonacci, subset sum)
- But naive recursion often recomputes the same sub-problems over and over
- This leads to exponential time in many cases

Fibonacci numbers

$F_n := F_{n-1} + F_{n-2}$ for $n > 2$
Base case: $F_1 := 1$ and $F_2 := 1$

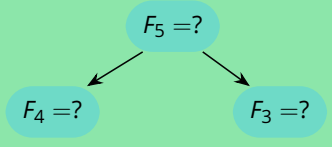


The Problem with Naive Recursion

- Many recursive definitions are straightforward (e.g. Fibonacci, subset sum)
- But naive recursion often recomputes the same sub-problems over and over
- This leads to exponential time in many cases

Fibonacci numbers

$F_n := F_{n-1} + F_{n-2}$ for $n > 2$
Base case: $F_1 := 1$ and $F_2 := 1$

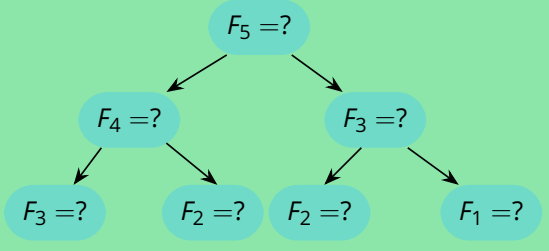


The Problem with Naive Recursion

- Many recursive definitions are straightforward (e.g. Fibonacci, subset sum)
- But naive recursion often recomputes the same sub-problems over and over
- This leads to exponential time in many cases

Fibonacci numbers

$F_n := F_{n-1} + F_{n-2}$ for $n > 2$
Base case: $F_1 := 1$ and $F_2 := 1$

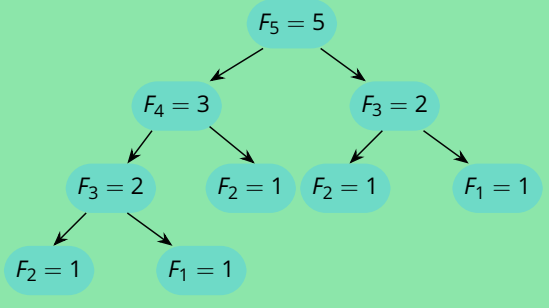


The Problem with Naive Recursion

- Many recursive definitions are straightforward (e.g. Fibonacci, subset sum)
- But naive recursion often recomputes the same sub-problems over and over
- This leads to exponential time in many cases

Fibonacci numbers

$F_n := F_{n-1} + F_{n-2}$ for $n > 2$
Base case: $F_1 := 1$ and $F_2 := 1$

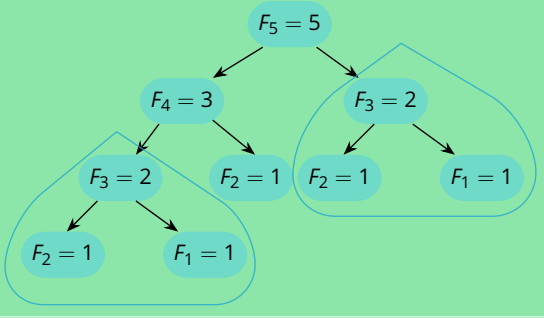


The Problem with Naive Recursion

- Many recursive definitions are straightforward (e.g. Fibonacci, subset sum)
- But naive recursion often recomputes the same sub-problems over and over
- This leads to exponential time in many cases

Fibonacci numbers

$F_n := F_{n-1} + F_{n-2}$ for $n > 2$
Base case: $F_1 := 1$ and $F_2 := 1$



Dynamic Programming (DP)

Definition

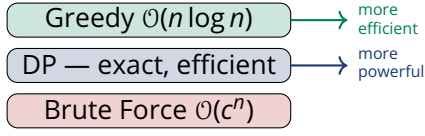
Dynamic Programming (DP) is an algorithm design paradigm that solves complex optimisation and counting problems by:

1. **Decomposing** the problem into simpler subproblems,
2. **Solving** each subproblem **exactly once**,
3. **Storing** results to avoid redundant recomputation.

Historical context

- Coined by Richard Bellman, 1950s
- Originally: optimal control theory
- Extended to: combinatorics, graph algorithms, bioinformatics, AI
- Prominent example: Dijkstra's algorithm to find shortest path

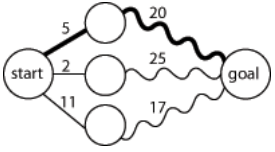
Paradigm hierarchy

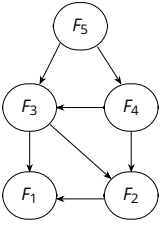


Two Necessary Conditions

Both conditions must hold simultaneously

1. **Optimal Substructure**
2. **Overlapping Subproblems**





Optimal Substructure

An optimal solution to the problem can be composed from optimal solutions to its subproblems.

Overlapping Subproblems

A naive recursive solution visits the **same subproblem instance** multiple times.

Condition 1: Optimal Substructure

Optimal Substructure

Let OPT(x) be the value of an optimal solution for instance x. The problem has **optimal substructure** if

$$\mathrm{OPT}(x) = f(\mathrm{OPT}(x_1), \mathrm{OPT}(x_2), \ldots, \mathrm{OPT}(x_k))$$

where $x_1, \ldots, x_k$ are strictly smaller instances.

Verification: Cut-and-Paste Argument

1. Take optimal solution S^* for instance x.
2. Suppose S^* uses sub-solution s for subproblem x_i.
3. Replace s with the true optimal s^* for x_i.
4. If the result is still **feasible** and **no worse** than S^* $\Rightarrow$ optimal substructure holds.
5. If feasibility or cost can be violated $\Rightarrow$ **DP cannot be applied directly**.

Optimal Substructure — Examples

Holds: Shortest Path (DAG)

Any subpath of a shortest path is itself a shortest path.

If $u \to v \to w$ is shortest, then $u \to v$ is shortest u-to-v.

Cut-and-paste: replacing $u \to v$ with a shorter route gives a shorter $u \to w$ — contradiction.

Holds: 0/1 Knapsack

Either item n is in S^* or it is not. In both cases, the remaining items are optimally selected for the remaining capacity.

Fails: Longest Simple Path

Graph: $u \to v \to w$, $u \to w$.

Longest simple path $u \to w$: $u \to v \to w$.

Longest simple path $u \to v$: $u \to v$.

But: $u \to v \to w$ uses w, so we cannot extend $u \to v$ through w again (no repeated vertices).

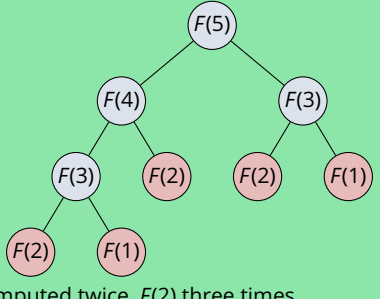
Sub-paths of longest simple paths are **not** longest simple paths.
NP-complete in general.

Condition 2: Overlapping Subproblems

Overlapping Subproblems

A problem has **overlapping subproblems** if a naive recursive solution visits the same subproblem instance **multiple times**.

Without memoisation: Fibonacci



$F(3)$ computed twice, $F(2)$ three times.
Tree size: $\mathcal{O}(\varphi^n)$ where $\varphi \approx 1.618$.

Contrast: Divide & Conquer

Merge Sort splits input into two halves. Each half appears in the recursion tree **exactly once**.

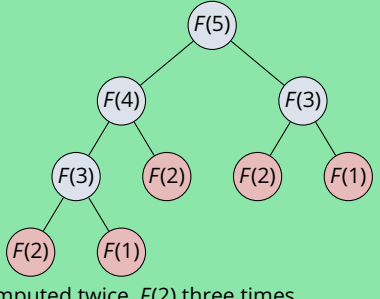
No two recursive calls share a subproblem ⇒ memoisation adds overhead without benefit ⇒ **DP is inapplicable**.

Condition 2: Overlapping Subproblems

Overlapping Subproblems

A problem has **overlapping subproblems** if a naive recursive solution visits the same subproblem instance **multiple times**.

Without memoisation: Fibonacci



$F(3)$ computed twice, $F(2)$ three times.
Tree size: $\mathcal{O}(\varphi^n)$ where $\varphi \approx 1.618$.

Rule of Thumb

Draw the recursion DAG. If it has **shared nodes** (a node reachable from multiple parents), overlapping subproblems exist.

Two Implementation Strategies

Dynamic Programming solves each sub-problem only once!

Two Implementation Strategies

Dynamic Programming solves each sub-problem only once!

Top-down approach (Memorization)

- Memorize solutions to sub-problems in a table (array, hashtable)
- When attempt to solve sub-problem first check table
- If contained in table use it directly, otherwise solve sub-problem and add to table

Two Implementation Strategies

Dynamic Programming solves each sub-problem only once!

Top-down approach (Memorization)

- Memorize solutions to sub-problems in a table (array, hashtable)
- When attempt to solve sub-problem first check table
- If contained in table use it directly, otherwise solve sub-problem and add to table

Bottom-up approach (Tabularization)

- Solve sub-problems first bottom up
- Use solutions to solve bigger sub-problems
- Tabular form to iteratively generating solutions to bigger and bigger sub-problems by using solutions to smaller sub-problems

Fibonacci Sequence

Naive implementation

function $fib(n)$
if $n <= 1$ **then**
  return n
else
  return $fib(n-1) + fib(n-2)$
end if=0

Fibonacci Sequence

Top-down approach

var $m := map(0 \to 0, 1 \to 1)$
function $fib(n)$
if key n is not in map m **then**
  $m[n] := fib(n-1) + fib(n-2)$
else
  return $m[n]$
end if=0

Uses O(n) time and space.

Fibonacci Sequence ctd.

Bottom-up approach

function *fib*(*n*)
if $n = 0$ **then**
  return 0
else
  var *previousFib* := 0, *currentFib* := 1
end if
for $n - 1$ times // loop is skipped if n = 1 **do**
  var *newFib* := *previousFib* + *currentFib*
  previousFib := *currentFib*
  currentFib := *newFib*
end for
return *currentFib* =0

Uses O(n) time but O(1) space.

Memoisation vs. Tabulation — Comparison

Dimension	Memoisation (top-down)	Tabulation (bottom-up)
Subproblems solved	Only reachable ones	All in defined space
Stack usage	$\mathcal{O}$(depth)	$\mathcal{O}(1)$ extra
Code structure	Mirrors recursive definition	Explicit fill order
Space optimisation	Harder	Straightforward (rolling array)
When to prefer	Sparse state space	Dense state space, known order

Key insight: asymptotic complexity is identical

Both strategies visit each subproblem once. The runtime formula is:

$$\mathcal{T}_{\text{DP}} = \underbrace{|\text{distinct subproblems}|}_{\text{state space size}} \times \underbrace{\text{work per subproblem}}_{\text{recurrence cost}}$$

Summary

- Search algorithms work on complete or partial solutions
- Exhaustive search is too expensive
- Heuristic functions estimate costs of shortest paths
- Good heuristics can dramatically reduce search cost
- Dynamic programming
  - complete and optimal
  - time and space consuming
  - how to define the sub-problems?

Applicability Checklist for DP

1. Optimal substructure? (cut-and-paste)
2. Overlapping subproblems? (recursion DAG)
3. Finite, manageable state space?
4. Well-defined base cases?

Summary

- Search algorithms work on complete or partial solutions
- Exhaustive search is too expensive
- Heuristic functions estimate costs of shortest paths
- Good heuristics can dramatically reduce search cost
- Dynamic programming
  - complete and optimal
  - time and space consuming
  - how to define the sub-problems?

Applicability Checklist for DP

1. Optimal substructure? (cut-and-paste)
2. Overlapping subproblems? (recursion DAG)
3. Finite, manageable state space?
4. Well-defined base cases?

Outlook

Next lecture: Local Search, Stochastic Hill Climbing, Simulated Annealing

References

- Zbigniew Michalewicz and David B. Fogel.
  How to Solve It: Modern Heuristics, volume 2. Springer, 2004.
- Stuart J. Russell and Peter Norvig.
  Artificial Intelligence - A Modern Approach (3. edition). Pearson Education, 2010.